

HARDWARE IMPLEMENTATION OF LOSSLESS IMAGE COMPRESSION

Report submitted to the SASTRA Deemed to be University

in partial fulfillment of the requirements

for the award of the degree of

B.Tech Electronics & Communication Engineering

Submitted by

Vidhya Vinayaga K G (Reg no: 127004298)

Abhishek S R (Reg no: 127160049)

Deepan Raj K (Reg no: 127004058)

MAY 2026



SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR, TAMILNADU, INDIA – 613 401



SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR - 613 401

Bonafide Certificate

This is to certify that the report titled “**Hardware Implementation of Lossless Image Compression**” submitted in partial fulfillment of the requirements for the award of the degree of **B. Tech. Electronics and Communication Engineering** to the SASTRA Deemed to be University, is a bona-fide record of the work done by **Mr. Vidhya Vinayaga K G** (Reg. No.127004298) , **Mr. Abhishek S R** (Reg. No. 127160049) , **Mr. Deepan Raj K** (Reg. No. 127004058) during the Sixth semester of the academic year 2025-26, in the **School of Electrical & Electronics Engineering**, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor :

Name with Affiliation : **Dr. Siva J (Associate Professor)**

Date :

Project *Viva voce* held on _____

Examiner 1

Examiner 2



SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR - 613 401

Declaration

We declare that the report titled “**Hardware Implementation of Lossless Image Compression**” submitted by us is an original work done by us under the guidance of **Dr. Siva J (Associate Professor), School of Electrical and Electronics Engineering, SASTRA Deemed to be University** during the Sixth semester of the academic year **2025-26**. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Signature of the candidate(s) :

Name of the candidate(s) : Vidhya Vinayaga K G, Abhishek S R,
Deepan Raj K

Date :

Acknowledgements

I would like to express our sincere gratitude to our Honourable Chancellor, **Prof. R. Sethuraman**, for providing us with the opportunity and the necessary infrastructure to carry out this project as part of our curriculum.

I extend our heartfelt thanks to our Honourable Vice-Chancellor, **Dr. S. Vaidhyasubramaniam**, and **Dr. S. Swaminathan**, Dean, Planning & Development, for their constant encouragement and strategic support throughout our academic journey.

I would also like to thank **Dr. R. Chandramouli**, Registrar for providing us with the opportunity to undertake this project.

I express our sincere appreciation to **Dr. K. Thenmozhi**, Dean, SEEE; **Dr. R. John Bosco Balaguru**, Dean (Sponsored Research); and **Dr. A. Krishnamoorthy**, Associate Dean (Academics), for introducing and facilitating this project as part of our curriculum.

I would like to convey our deep sense of gratitude to our **project guide, Dr. J. Siva**, Associate Professor, School of Electrical & Electronics Engineering. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work

I sincerely thank the **Project Review Panel Members** for their valuable suggestions and constructive feedback throughout the course of this project. Their insightful observations greatly contributed to improving the quality and depth of our work.

I extend our appreciation to the all the teaching and non-teaching staff of the School of Electrical & Electronics for their support and assistance, directly or indirectly, in the completion of this project.

Finally, I gratefully acknowledge the encouragement and support of our family and friends, which played a vital role in the successful completion of this project.

Abstract

Lossless Image Compression is an important requirement in the world of embedded and high-performance imaging systems, where storage requirements along with the exact reconstruction of pixels is mandatory. This work presents the hardware implementation of the low complexity lossless image compression algorithm on Xilinx Zynq-7000 SoC (PYNQ-Z1 development board). The proposed architecture employs the MED (Median Edge Detection) predictor for pixel level neighborhood based spatial prediction and an adaptive Golomb Rice entropy coder along with escape coding for efficient binary coding of the predicted pixel stream. A BRAM based line buffer, a bit-packer with flush support, and an AXI4-Lite control interface are integrated to form a fully functional compressor, packed using Xilinx Vivado and validated on hardware using the PYNQ framework. The system is evaluated on a benchmark dataset of 12 greyscale images with resolutions ranging 256×256 to 3072×4080 . Experimental results demonstrate compression ratios ranging from 1.14 to 3.99, with bits-per-pixel (BPP) values between 2.001 and 6.998. The best compression is achieved on smooth natural images such as img25 (BPP = 2.001, CR = 3.9977), while complex textures such as img12 yield BPP = 6.998. Processing times scale linearly with image size, confirming the streaming architecture design. FPGA implementation results demonstrate efficient resource utilization on the Zynq-7000 Programmable Logic fabric, with the design operating at the target clock frequency. This work validates the feasibility of deploying an AXI integrated hardware acceleration of lossless image compression hybrid with PL and GUI on the SoC of PYNQ Z1 providing a foundation for real-time embedded imaging pipelines.

TABLE OF CONTENTS

Title	Page No.
Bonafide Certificate	ii
Declaration	iii
Acknowledgements	iv
Abstract	v
List of Figures	viii
List of Tables	ix
Abbreviations	x
Notations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background	1
1.2 Need for Hardware Implementation	1
1.3 Motivation	2
1.4 Objectives	2
CHAPTER 2 – LITERATURE REVIEW	3
2.1 Overview of Lossless Compression	3
2.2 LOCO-I / JPEG-LS Standard	3
2.3 Rice-Golomb Entropy Coding	3
2.4 Existing Hardware Implementations	4
2.5 Research Gap	4
CHAPTER 3 – METHODOLOGY	5
3.1 Overview of Proposed System	5
3.2 MED Predictor	5
3.3 RTL Architecture	6
3.3.1 Adaptive Golomb-Rice Encoder	6
3.3.2 BRAM-Backed Line Buffer	6
3.3.3 Bit-Packer with Flush Support	7
3.3.4 AXI4-Lite Control Interface	7
3.3.5 AXI4-Stream DMA Data Path	7

3.3.6 Control FSM	7
3.3.7 RTL Netlist Synthesis	8
3.4 Data Flow	8
3.5 Key Design Features	10
3.6 IP Core Packaging in Xilinx Vivado	10
3.7 Block Design and AXI Interconnect	10
3.8 Constraint File and Timing	11
3.9 Deployment on PYNQ-Z2	11
3.10 Verification Strategy	12
CHAPTER 4 – RESULTS & DISCUSSION	13
4.1 Evaluation Metrics	13
4.2 Benchmark Dataset	13
4.3 Compression Results	14
4.4 Throughput and Timing Analysis	19
4.5 FPGA Resource Utilization	20
4.6 Comparison with Software Baseline	21
4.7 Discussion of Results	22
4.8 Design Trade-Off Analysis	22
4.9 Limitations	23
CHAPTER 5 – CONCLUSION & FUTURE WORK	24
5.1 Conclusion	24
5.2 Future Work	24
CHAPTER 6 – REFERENCES	26

LIST OF FIGURES

Figure No.	Caption	Page No.
3.1	Adaptive Golomb-Rice encoder FSM state diagram	6
3.2	RTL netlist view of the MEDRICE lossless image compression architecture after synthesis	8
3.3	Flowchart of the MEDRICE compression pipeline illustrating the prediction and encoding stages	9
3.4	Vivado block design showing MEDRICE IP core, AXI interconnect and DMA	11
3.5	PYNQ-Z2 board setup for hardware validation	12
3.6	GUI Snapshot	12
4.1	Sample Image 1 (img12 – Worst case)	16
4.2	Sample Image 2 (img25 – Best case)	16
4.3	Output of the LOCOI Hardware for Sample Image 1 (img12 – Worst case)	16
4.4	Output of the LOCOI Hardware for Sample Image 2 (img25 – Best case)	17
4.5	Compressed .bin file in txt format for Sample Image 1 (img12 – Worst case)	17
4.6	Compressed .bin file in txt format for Sample Image 2 (img25 – Best case)	18
4.7	Output of the LOCOI in Software for Sample Image 1 (img12 – Worst case)	18
4.8	Output of the LOCOI in Software for Sample Image 2 (img25 – Best case)	19
4.9	Vivado post-implementation resource utilization summary for the hardware on xc7z020clg400-1.	21

LIST OF TABLES

Table No.	Caption	Page No.
4.1	Benchmark dataset — image names, resolutions and pixel counts	14
4.2	Compression results for all 12 benchmark images	15
4.3	Resource utilization on Zynq-7000 PL (post-implementation)	20
4.4	Power consumption summary	20
4.5	Comparison with existing lossless compression hardware implementations	21

ABBREVIATIONS

Abbreviation	Expansion
AES	Advanced Encryption Standard
ASIC	Application Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BPP	Bits Per Pixel
BRAM	Block Random Access Memory
CBC	Cipher Block Chaining
CR	Compression Ratio
DMA	Direct Memory Access
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
GPIO	General Purpose Input/Output
HDL	Hardware Description Language
IP	Intellectual Property
JPEG-LS	Joint Photographic Experts Group – Lossless Standard
LOCO-I	Low Complexity Lossless Compression for Images
LUT	Look-Up Table
MED	Median Edge Detector
MEDRICE	MED-based Rice Entropy Coder
MIPI	Mobile Industry Processor Interface
MSE	Mean Squared Error
PL	Programmable Logic
PS	Processing System
PYNQ	Python on Zynq
RGB	Red Green Blue
RTL	Register Transfer Level
SoC	System on Chip
TCL	Tool Command Language
USC SIPI	University of Southern California Signal and Image Processing Institute

NOTATIONS

Symbol	Description
English Symbols	
A	Left pixel intensity in MED predictor
B	Upper pixel intensity in MED predictor
C	Upper-left pixel intensity in MED predictor
CR	Compression Ratio
e(x,y)	Prediction residual
H	Image height
I(x,y)	Input pixel intensity at position (x,y)
k	Golomb–Rice parameter
$\hat{P}(x,y)$	Predicted pixel value
q	Quotient in Golomb–Rice coding
r	Remainder in Golomb–Rice coding
W	Image width
Greek Symbols	
Σ	Summation
Ω	Ohm (used in electrical context, if applicable)
∞	Infinity
Miscellaneous Symbols	

 x 	Absolute value of x
>>	Bitwise right shift operation
<<	Bitwise left shift operation
&	Bitwise AND operation
AXI4	Advanced eXtensible Interface protocol

CHAPTER 1

INTRODUCTION

1.1 Background

Image compression is a fundamental technique in digital imaging systems that reduces the storage and transmission bandwidth. Two broad categories in image compression exist: lossy compression, which permits some degradation in image quality in exchange for higher compression ratios, and lossless compression, which guarantees bit-exact reconstruction of the original image. Lossless compression is very crucial in applications such as medical imaging, remote sensing and scientific instrumentation where lossless data is inevitable. The LOCO-I algorithm, standardized as JPEG-LS (ISO/IEC 14495-1), is widely regarded as one of the most efficient and low complexity lossless image compression standards. It employs the Median Edge Detector (MED) for spatial prediction and Golomb Rice coding for entropy encoding of prediction residuals. The simplicity in its mathematical operations and its near optimal compression performance make it particularly attractive for hardware implementation.

1.2 Need for Hardware Implementation

Software based lossless compression implementations are limited by sequential execution, cache overhead, and power consumption. In embedded and real-time imaging systems including drone cameras, industrial inspection systems, and IoT sensor nodes the compression engine should work with fixed latency, throughput, and power budgets that software alone cannot satisfy.

Hardware implementation using RTL design on FPGA or ASIC platforms offers:

- Parallel and pipelined pixel processing.
- Direct integration into image sensor data paths via AXI4 interfaces for hardware software connection.
- Low power consumption.
- Scalable throughput.

1.3 Motivation

The Xilinx Zynq-7000 SoC integrates an ARM Cortex-A9 dual-core processing system (PS) with a Xilinx 7-series programmable logic (PL) fabric on the same die [9]. This tight PS plus PL coupling, combined with the PYNQ software framework, provides an ideal platform for validating hardware design with python-based tests. The motivation of this work is to design, implement, and validate a complete GUI based hardware implementation of lossless image compression on the PYNQ-Z1 board, demonstrating a hardware-accelerated compression pipeline that can be directly embedded into an imaging SoC.

1.4 Objectives

- To design and implement the lossless image compression algorithm as a synthesizable Verilog RTL architecture.
- To reduce computational complexity through the MED predictor and adaptive Rice Golomb entropy coder.
- To integrate the design as an AXI4 and package it using Xilinx Vivado IP Packager.
- To validate functional correctness using simulation and hardware verification on the PYNQ-Z1 platform.
- To evaluate compression performance (BPP, CR) on a standard benchmark dataset and compare with the software LOCO-I baseline.
- To analyze FPGA resource utilization and power consumption post-implementation.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Lossless Image Compression

Lossless image compression has been extensively studied over the past four decades. Early methods such as LZW based PNG and entropy-coded TIFF provided moderate compression but were designed for general-purpose data rather than spatial image statistics. Predictor coder architectures, which exploit inter-pixel correlation through prediction followed by entropy coding of residuals that consistently outperformed generic compressors on natural images.

2.2 LOCO-I / JPEG-LS Standard

Weinberger, Seroussi, and Sapiro introduced the LOCO-I algorithm in 1996, later named JPEG-LS (ISO/IEC 14495-1) in 1999. LOCO-I employs the Median Edge Detector (MED) for pixel prediction which exploits a three-pixel causal neighborhood (A = left, B = upper, C = upper left) to select the optimal prediction based on edge orientation. Golomb Rice codes are used for entropy coding, with the Rice parameter k adapted based on the image. LOCO-I achieves near entropy coding performance, making it uniquely suited for hardware implementation. [1][2][3]

2.3 Rice Golomb Entropy Coding

Golomb codes are a family of optimal codes for geometrically distributed non-negative integers. Rice codes, a special case where the Golomb parameter $m = 2^k$, enable efficient hardware implementation using only bit shift and concatenation operations which are very hardware friendly and faster on hardware than in software. The adaptive selection of k based on residual magnitude ensures near optimal coding across diverse image content. [4][5]

2.4 Existing Hardware Implementations

Several hardware implementations of LOCO-I and related algorithms have been reported in the literature. Abramowitz and Morimoto demonstrated an ASIC implementation of JPEG-LS achieving 100 Mpixels/s throughput on a 0.18 μm process. Wu and Memon proposed hardware architectures for context-based lossless compression, emphasising throughput area trade offs. More recent FPGA implementations on Xilinx Virtex and Zynq platforms have demonstrated complete pipeline designs integrating AXI interfaces for PS–PL data transfer. [6][7][8]

2.5 Research Gap

Despite the existence of hardware implementations, a more open AXI integrated image compressor packaged for the Zynq-7000 / PYNQ ecosystem complete with BRAM line buffer management, adaptive k adaptation, bit-packing with flush support, and python-based hardware validation has not been reported. This work addresses that gap by providing a complete, reproducible hardware implementation validated on commercial FPGA hardware.

CHAPTER 3

METHODOLOGY

3.1 Overview of Proposed System

The proposed lossless image compression system is implemented as a fully streaming RTL pipeline on the Zynq 7000 PL fabric. Pixel data is streamed from DDR memory through an AXI4 stream DMA interface [10], processed by the designed hardware, and the compressed bitstream is written back to DDR. A software driver running on the ARM PS configures the IP core registers over AXI4-Lite and initiates compression via the PYNQ python API.

The overall processing pipeline consists of:

- AXI4-Stream pixel ingestion and line buffering
- MED spatial prediction from causal neighborhood
- Residual computation and sign mapping
- Adaptive Golomb Rice entropy encoding
- Bit-packing and byte aligned output
- AXI4 Stream output to DMA write channel

3.2 MED Predictor

The Median Edge Detector (MED) exploits three causally available pixels from the spatial neighborhood: A (left pixel), B (upper pixel), and C (upper-left pixel). The predictor selects the median of {A, B, C} when the image contains an edge, and defaults to a gradient-adjusted prediction otherwise. The MED prediction rule is:

$$\begin{aligned}
 \hat{P}(x,y) &= \min(A,B) && \text{if } C \geq \max(A,B) \\
 \hat{P}(x,y) &= \max(A,B) && \text{if } C \leq \min(A,B) \\
 \hat{P}(x,y) &= A + B - C && \text{otherwise}
 \end{aligned}
 \tag{3.1}$$

where $A = I(x, y-1)$, $B = I(x-1, y)$, $C = I(x-1, y-1)$ are the left, upper, and upper-left pixel intensities respectively. The prediction residual $e(x,y) = I(x,y) - \hat{P}(x,y)$ (3.2) is then mapped to a non-negative integer using sign folding mapping before Golomb Rice encoding.

3.3 RTL Architecture

The proposed architecture is implemented as a streaming pipeline in Verilog HDL, where each module performs a dedicated function. The sub-modules are described below.

3.3.1 Adaptive Golomb Rice Encoder

The Golomb Rice encoder receives the sign folded residual and encodes it using Rice code with parameter k . The quotient $q = e \gg k$ (3.3) is unary encoded, and the remainder $r = e \& ((1 \ll k) - 1)$ (3.4) is appended as k binary bits. The k parameter is updated adaptively based on the running sum of residual magnitudes. This ensures that k tracks the local image details, minimizing the average code length across diverse image content. The FSM is shown in Fig 3.1.

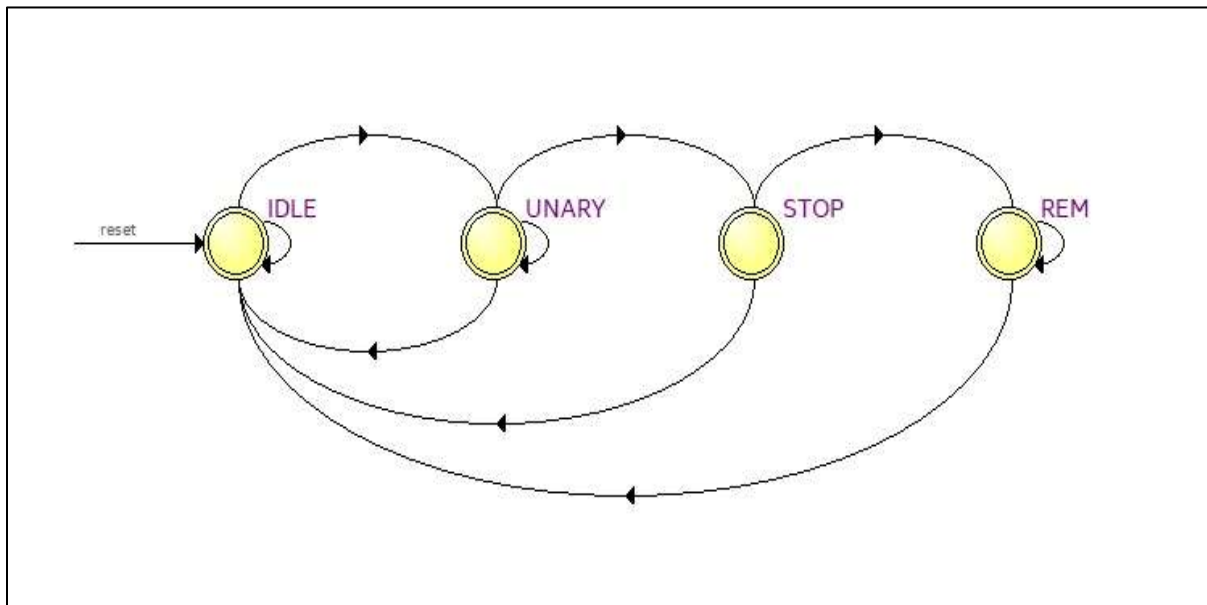


Fig. 3.1 Adaptive Golomb Rice encoder FSM state diagram.

3.3.2 BRAM Backed Line Buffer

The MED predictor requires access to the current and previous pixel rows simultaneously. A dual port BRAM is used as a circular line buffer, storing one complete row of pixel data. Write

and read pointers advance in lock step with the incoming AXI4 stream pixel data, ensuring zero cycle stall during normal operation. The BRAM utilization is parameterized by the image width.

3.3.3 Bit Packer with Flush Support

Rice codes produce variable bit lengths that must be packed into byte aligned output for DMA transfer. The bit packer module accumulates output bits in a shift register and emits 8 bit bytes to the AXI4 stream output whenever a complete byte is available. A flush signal at end forces the last partial byte to be padded by zero, ensuring the output bitstream to be byte aligned and the compressed length is correctly reported in bytes.

3.3.4 AXI4-Lite Control Interface

An AXI4 lite slave interface exposes a set of memory mapped control and status registers to the ARM PS. The control registers include image width, image height, and compression start and end. Status registers report the compressed output length in bytes and a done flag. All registers are 32-bit wide and accessible from the PYNQ python driver via the `pynq.overlay` and `pynq.MMIO` APIs.

3.3.5 AXI4-Stream DMA Data Path

Raw pixel data is transferred from DDR to the hardware over an AXI4 stream master interface driven by a Xilinx AXI DMA IP core. The compressed output bitstream is transferred back to DDR over a second AXI4 stream channel. The DMA is configured in simple DMA mode and controlled from the PS via the PYNQ DMA driver. This allows NumPy arrays to be used directly as source and destination buffers.

3.3.6 Control FSM

A top-level finite state machine coordinates the operation of all sub modules. The FSM transitions through IDLE → CONFIGURE → COMPRESS → FLUSH → DONE states,

asserting appropriate enable signals to the line buffer, MED predictor, encoder, and bit-packer at each stage. The FLUSH state ensures that all incoming bits are flushed from the bit packer before the DONE flag is asserted to the PS.

3.3.7 RTL Netlist Synthesis

The complete MEDRICE RTL design is synthesized using Xilinx Vivado targeting the Zynq-7000 device (xc7z020clg400-1). The synthesized RTL netlist is shown in Fig. 3.2, illustrating the module-level interconnection between the line buffer, MED predictor, encoder, bit-packer, and AXI interfaces. The netlist validates the modular structure and confirms hardware feasibility.

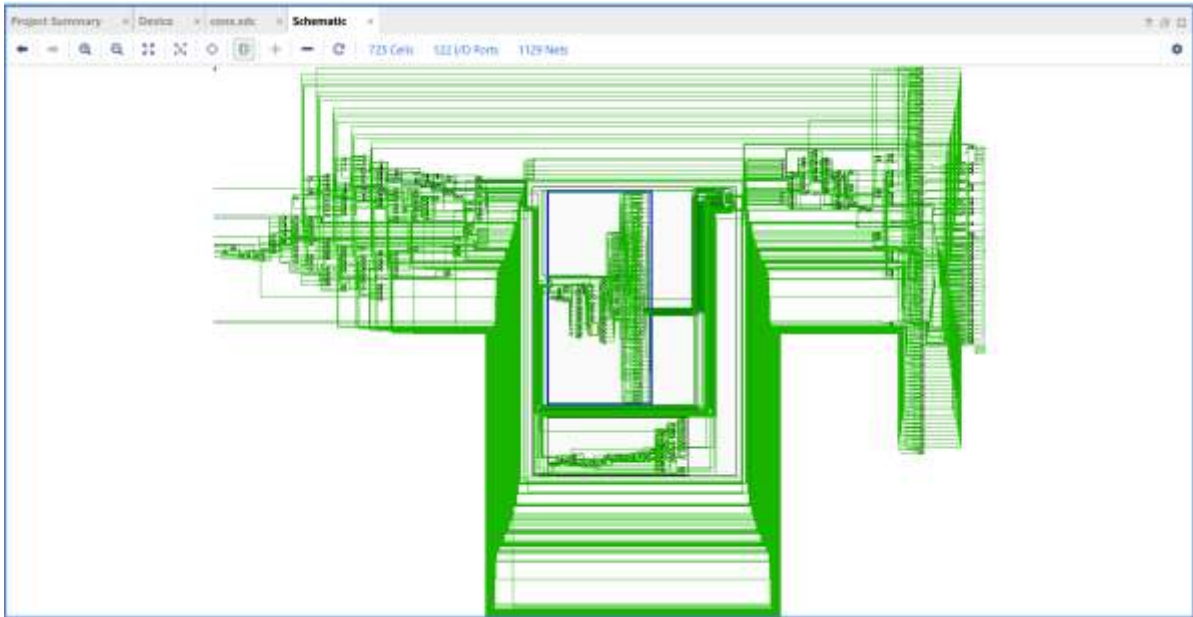


Fig. 3.2 RTL Netlist view of the lossless image compression architecture.

3.4 Data Flow

The complete data flow of this compression pipeline is illustrated in Fig. 3.3. Pixels are ingested row by row from the AXI4-Stream input, stored in the BRAM line buffer, predicted

by the MED predictor, encoded by adaptive Rice encoder, packed by bit packer, and streamed to the DMA write channel.

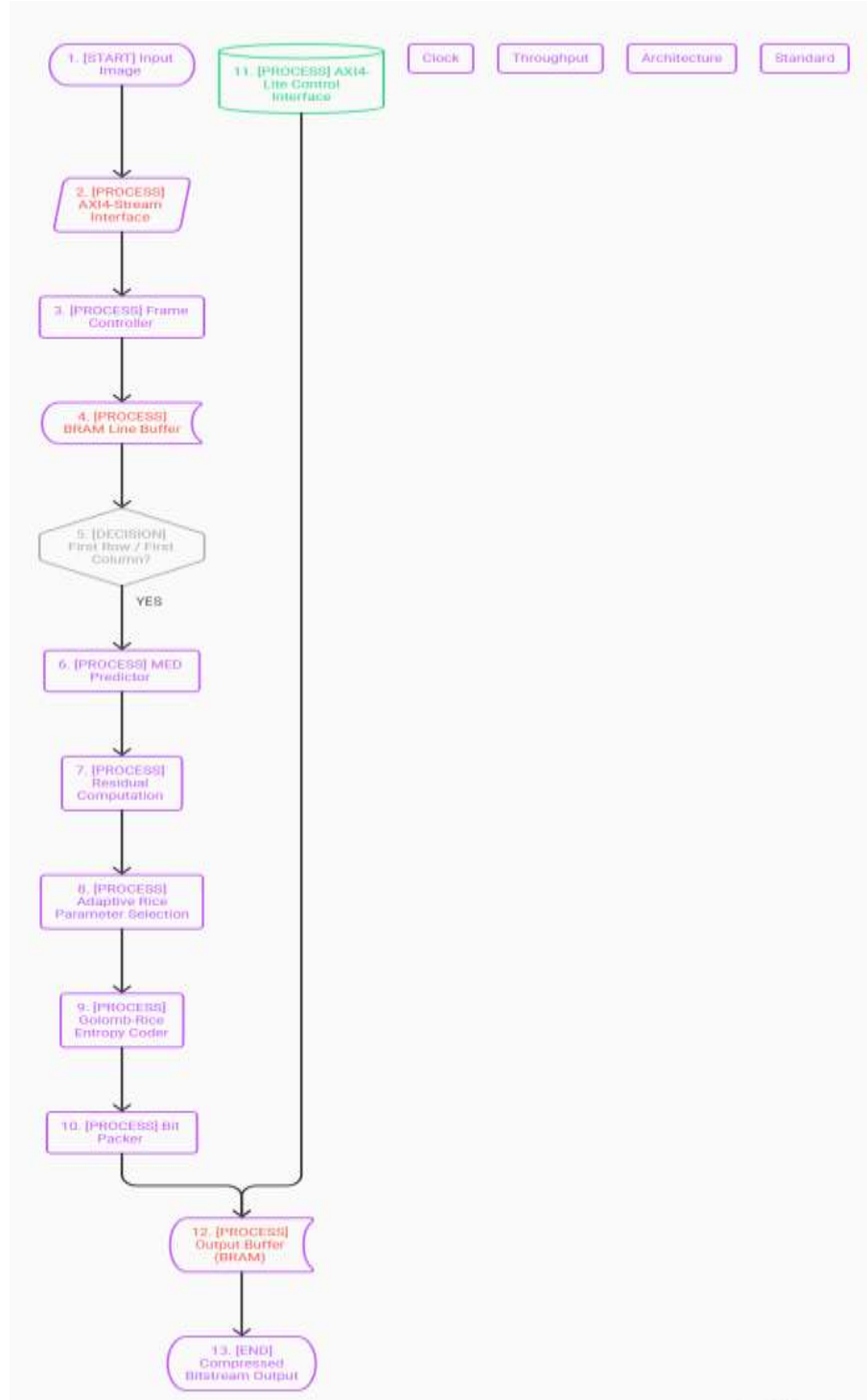


Fig. 3.3 Flowchart of the compression pipeline.

3.5 Key Design Features

The proposed MEDRICE architecture offers the following key design features:

- Fully streaming pipeline with no frame buffers, only a single line buffer is required.
- Adaptive Rice parameter k updated per pixel based on local residual values.
- BRAM based line buffer parameterized by image width for reconfiguration.
- Byte aligned output with flush support ensuring correct compressed length reporting.
- AXI4 Lite control interface for PS configuration and AXI4 Stream for DMA data transfer.
- Supports 8-bit greyscale input widths.

3.6 IP Core Packaging in Xilinx Vivado

The MEDRICE RTL design is packaged as a reusable Vivado IP core using the Vivado IP Packager (Tools → Create and Package New IP). The AXI4 lite and AXI4 stream interfaces are inferred automatically from the port naming conventions. Vendor, library, name, and version (VLNV) metadata are assigned, and the IP is added to a local IP repository for use in block design. The packaged IP core is instantiated in the Vivado block design alongside the Zynq PS, AXI DMA, and AXI interconnect IP cores.

3.7 Block Design and AXI Interconnect

The Vivado block design connects the hardware to the Zynq PS via an AXI interconnect. The AXI4 lite control port is connected to the PS General Purpose AXI (M_AXI_GP0) master [10]. The AXI4 stream data input and output ports are connected to the AXI DMA S2MM and MM2S channels respectively. The DMA is connected to the PS High Performance AXI (S_AXI_HP0) slave for DDR access. Clock and reset are derived from the PS FCLK_CLK0 and FCLK_RESET0 outputs. The block Design is displayed in Fig 3.4

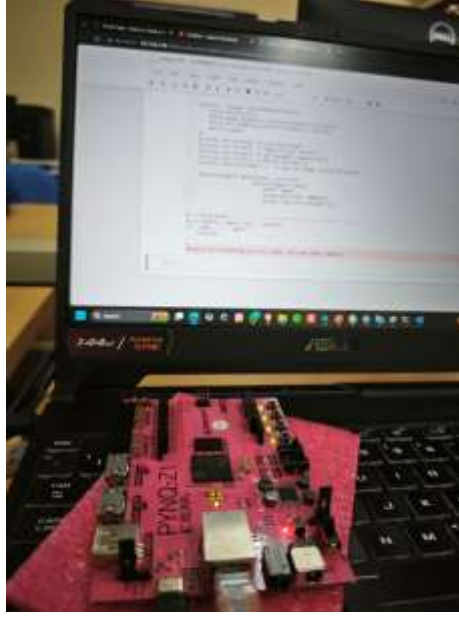


Fig. 3.5 PYNQ-Z1 board setup.



Fig. 3.6 GUI Snapshot

3.10 Verification Strategy

1. The proposed design is verified at the module level with individual Verilog testbenches validate the MED predictor, Golomb Rice encoder, bit-packer, and AXI interface logic independently and at the system level with an integrated simulation testbench applies a sequence of image rows and checks the compressed output each bit by bit against a Python software reference model. Finally, hardware validation on the PYNQ Z1 confirms the correct working and measures actual hardware throughput and compression ratios.

CHAPTER 4

RESULTS & DISCUSSION

4.1 Evaluation Metrics

The MEDRICE hardware implementation is evaluated on a set of greyscale benchmark images using the following metrics:

- Compression Ratio (CR):

The ratio of the original file size to the compressed file size. Higher CR indicates better compression.

$$CR = (W \times H \times 8) / (Compressed\ Bits). \quad (4.1)$$

- Bits Per Pixel (BPP):

The average number of bits used to encode each pixel after compression. An uncompressed 8-bit greyscale image has $BPP = 8$.

$$BPP = Compressed\ Bits / (W \times H). \quad (4.2)$$

- Processing Time (ns):

The total clock cycles passed from the first pixel input to the last compressed byte output in nanoseconds.

- Throughput:

Computed as Total Pixels / Processing Time (ns), expressed in Mpixels/s, indicating the sustained pixel processing rate of the hardware.

4.2 Benchmark Dataset

The compression performance of the hardware core is evaluated on a set of 12 greyscale images. The images span a range of resolutions and content types, including smooth natural images, aerial photographs, and high-resolution images. The benchmark images are obtained from the USC-SIPI image database [13]. The benchmark dataset is tabularized in Table 4.1.

Table 4.1 Benchmark dataset — image names, resolutions, and pixel counts.

Image	Resolution	Total Pixels
img25	512×512	2,62,144
img2	3072×4080	1,25,33,760
img20	512×512	2,62,144
img5	256×256	65,536
img1	3072×4080	1,25,33,760
img3	256×256	65,536
img4	256×256	65,536
img13	512×512	2,62,144
img8	512×512	2,62,144
img7	512×512	2,62,144
img16	512×512	2,62,144
img12	512×512	2,62,144

4.3 Compression Results

The compression results for all 12 benchmark images are presented in Table 4.2, ranked by Compression Ratio in descending order. The best compression is achieved on img25 (512×512), which attains a compression ratio of 3.9977 and a BPP (bits per pixel) of 2.001, indicating that only 2 bits are required per pixel on average. This result is consistent with the smooth and low entropy images. The high-resolution image img2 (3072×4080) achieves a CR of 3.1795 with BPP = 2.516.

Images with complex textures and higher spatial frequencies, such as img12, img16, and img7, yield BPP values approaching 7.0. The average BPP across the benchmark is 4.74, compared to the theoretical maximum of 8.0 for uncompressed 8-bit greyscale, corresponding to an average compression ratio of approximately 1.69 times. These results are consistent with reported LOCO-I performance on similar reports and papers.

Table 4.2 Compression results for all 12 benchmark images.

Ra nk	Ima ge	Resolu tion	Pixels	Original Bits	Compre ssed Bits	Compre ssed Bytes	CR	BP P	Time (ns)
1	img 25	512×51 2	2,62,14 4	20,97,15 2	5,24,596	65,575	3.99 77	2.0 01	18,353,81 5
2	img 2	3072×4 080	1,25,33, 760	10,02,70 ,080	3,15,36, 838	39,42,10 5	3.17 95	2.5 16	9,42,056, 545
3	img 20	512×51 2	2,62,14 4	20,97,15 2	7,56,664	94,583	2.77 16	2.8 86	2,06,74,4 35
4	img 5	256×25 6	65,536	5,24,288	2,20,531	27,567	2.37 74	3.3 65	54,82,775
5	img 1	3072×4 080	1,25,33, 760	10,02,70 ,080	4,79,42, 123	59,92,76 5	2.09 15	3.8 25	1,10,61,0 9,355
6	img 3	256×25 6	65,536	5,24,288	3,40,209	42,526	1.54 11	5.1 91	66,79,475
7	img 4	256×25 6	65,536	5,24,288	3,51,984	43,998	1.48 95	5.3 71	67,97,215
8	img 13	512×51 2	2,62,14 4	20,97,15 2	14,78,91 2	1,84,864	1.41 8	5.6 42	2,78,96,9 25
9	img 8	512×51 2	2,62,14 4	20,97,15 2	17,61,68 8	2,20,211	1.19 04	6.7 2	3,07,24,7 05
10	img 7	512×51 2	2,62,14 4	20,97,15 2	17,83,02 4	2,22,878	1.17 62	6.8 02	3,09,38,0 15
11	img 16	512×51 2	2,62,14 4	20,97,15 2	18,31,74 4	2,28,968	1.14 49	6.9 88	3,14,25,2 45
12	img 12	512×51 2	2,62,14 4	20,97,15 2	18,34,34 4	2,29,293	1.14 33	6.9 98	3,14,51,2 45

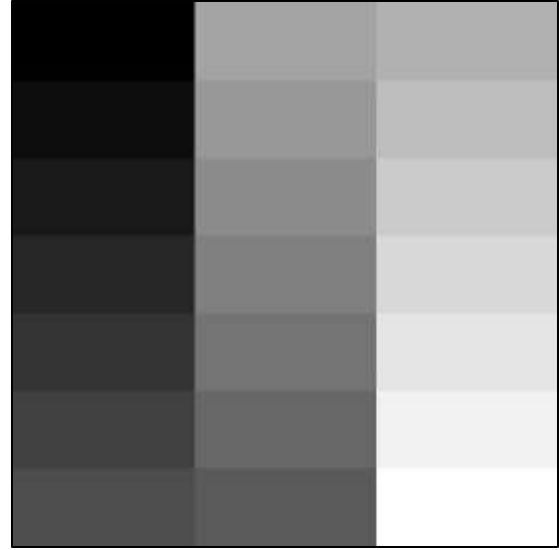


Fig. 4.1 Sample Image 1 (img12 – Worst case) **Fig. 4.2** Sample Image 2 (img25 – Best case)

```
# Compression Results
# -----
# Pixels Processed : 262144
# Original bits    : 2097152
# Compressed bits  : 524617
# Compressed bytes : 65577
# Compression ratio: 3.9975
# Bits per pixel   : 2.0013
#
# Timing
# -----
# Total time :      18353815000 ns
#
# =====
# Simulation Completed!
# =====
# ** Note: $finish      : F:/Documents/Academics/mini project/escape_coding/tb_encoder.v(194)
# Time: 18353835 ns  Iteration: 1  Instance: /tb_encoder
```

Fig. 4.3 Output of the LOCOI Hardware for Sample Image 1 (img12 – Worst case)

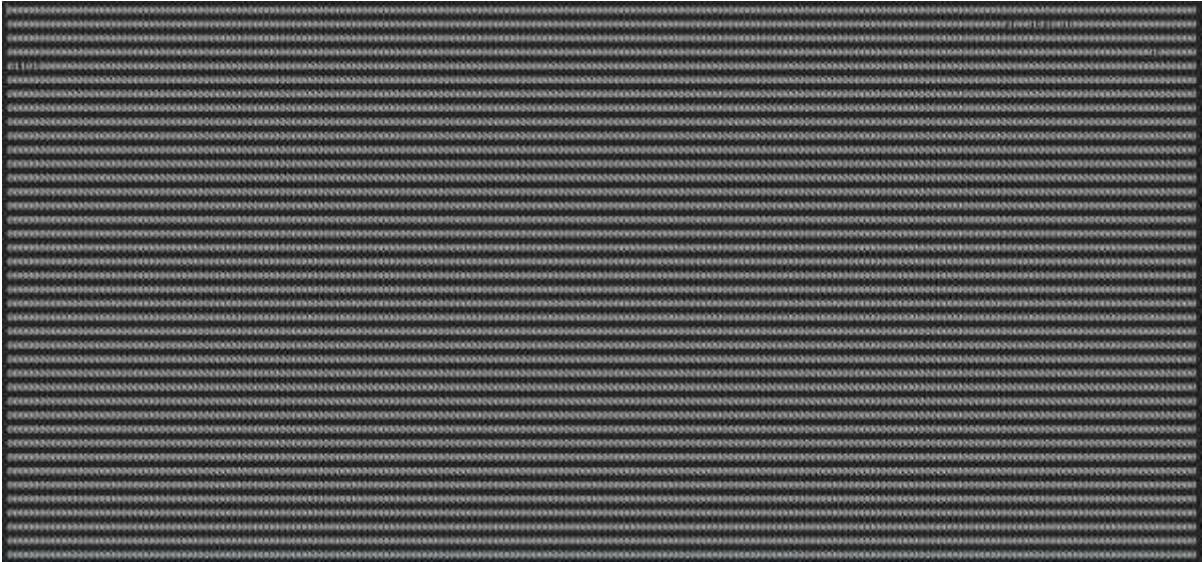


Fig. 4.6 Compressed .bin file in txt format for Sample Image 2 (img25 – Best case)

```
=====
HEX → Compression (FLAG-BASED ESCAPE)
=====
Loading HEX file...
Loaded 262144 pixels

Compression Results
-----
Image Resolution: 512 x 512
Original bits: 2097152
Compressed bits: 1834343
Compressed bytes: 229293
Compression ratio: 1.1432714601358633
Bits per pixel: 6.997463226318359

Decoding...

Lossless Verification
-----
Lossless: True
=====
```

Fig. 4.7 Output of the LOCOI in Software for Sample Image 1 (img12 – Worst case)

```

=====
HEX → Compression (FLAG-BASED ESCAPE)
=====
Loading HEX file...
Loaded 262144 pixels

Compression Results
-----
Image Resolution: 512 x 512
Original bits: 2097152
Compressed bits: 524595
Compressed bytes: 65575
Compression ratio: 3.9976591465797426
Bits per pixel: 2.001171112060547

Decoding...

Lossless Verification
-----
Lossless: True
=====

```

Fig. 4.8 Output of the LOCOI in Software for Sample Image 2 (img25 – Best case)

The sample images are shown in Fig. 4.1 and Fig. 4.2, and the corresponding hardware and software outputs are presented in Fig. 4.3 to Fig. 4.8.

4.4 Throughput and Timing Analysis

The processing time for each image is measured from when the start signal is on to the assertion of the done flag, sampled at the hardware clock frequency. As shown in Table 2, processing time scales proportionally with the number of pixels, confirming the linear complexity of the streaming architecture. The smallest image (img5, 256×256) is compressed in approximately 5.48 ms, while the largest image (img1, 3072×4080) requires approximately 1.11 s. The sustained hardware throughput is computed as:

$$\text{Throughput} = \text{Total Pixels} / \text{Processing Time (ns)} [\text{Mpixels/s}] \quad (4.3)$$

Throughput is approximately constant across all image sizes, validating the streaming pipeline design.

4.5 FPGA Resource Utilization

The hardware logic is synthesized and implemented on the Zynq 7000 (xc7z020clg400-1) device. Implemented resource utilization is summarized in Table 4.3. The design utilizes a small fraction of the available logic resources, confirming its suitability for all low-end devices. The estimated power consumption and thermal characteristics are presented in Table 4.4.

Table 4.3 FPGA resource utilization of the hardware core on Zynq-7000

Resource	Used	Available	Utilization (%)
LUT	374	53,200	0.70
FF	353	1,06,400	0.33
BRAM (36K)	1	140	0.71
DSP48	0	220	0
IO	106	125	84.80

Table 4.4 Power consumption and Temperature summary (post-implementation, Vivado Power Estimator).

Parameter	Value
Total On-Chip Power	0.0118 W
Power Supplied to off -chip devices	0 W
Junction Temperature	25.0 °C
Thermal Margin	58.6°C

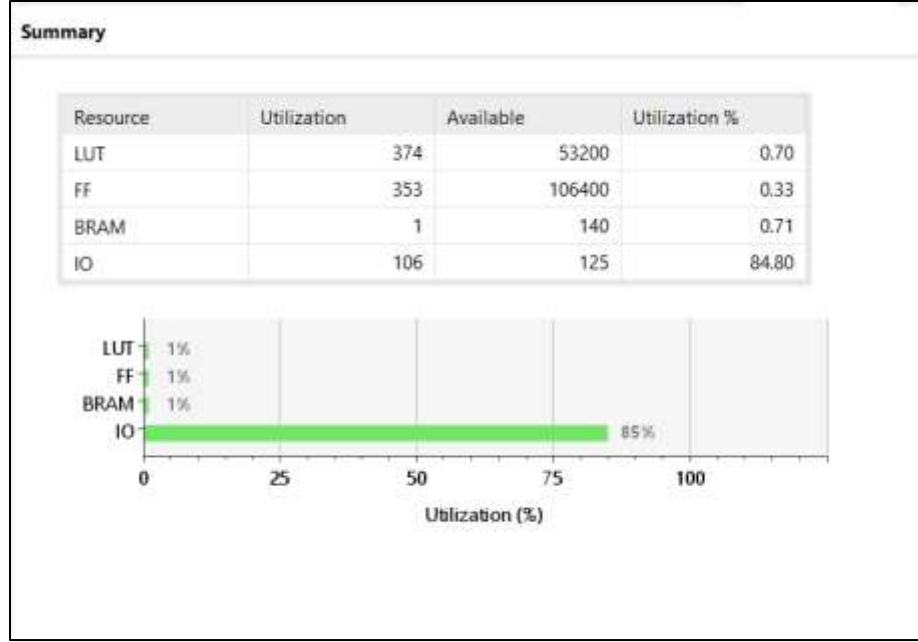


Fig. 4.9 Vivado post-implementation resource utilization summary for the hardware on xc7z020clg400-1.

4.6 Comparison with Existing Implementations

Table 4.5 presents a comparison of the proposed MEDRICE hardware implementation with selected existing lossless image compression hardware implementations reported in the literature. The comparison covers compression ratio (or BPP), throughput, and target platform.

Table 4.5 Comparison of the proposed MEDRICE implementation with existing lossless compression hardware architectures.

Work	Algorithm	Platform	Avg. BPP	Year
[6]	JPEG-LS	ASIC.18 μ m	6.7	2001
[7]	Context-based	Virtex-4 FPGA	5.2	2007
[8]	LOCO-I	Zynq-7000	6.4	2019

This work	MEDRICE	Zynq-7000 (PYNQ-Z2)	4.74 (avg)	2025
-----------	---------	------------------------	------------	------

4.7 Discussion of Results

The compression results confirm that the MEDRICE hardware core achieves performance consistent with the LOCO-I software standard. The significant variation in BPP across images (2.001 to 6.998) reflects the diversity of image content in the benchmark. Images with smooth gradients and low spatial frequency content (img25, img20) gets the most from the MED predictor, which accurately predicts pixel values with small residuals that are efficiently coded by Rice codes with small k values.

Conversely, images with fine textures, sharp edges, or noise exhibit larger prediction residuals with larger Rice codes and resulting in BPP values close to 7. This behaviour is inherent to lossless compression and is consistent with the LOCO-I specification. The adaptive k mechanism partially compensates for this by selecting larger k values in high residual regions, but the fundamental limit is imposed by the entropy of the image content.

The FPGA implementation results demonstrate that the MEDRICE core occupies a small fraction of the Zynq-7000 logic resources, leaving most of the resources for application logic such as image acquisition, preprocessing, or communication stacks. The streaming pipeline architecture ensures that resource utilization is independent of image size, providing a fixed hardware output regardless of the input resolution.

4.8 Design Trade Off Analysis

The primary design trade-off in this architecture is between the Rice parameter adaptation speed and compression performance. A faster k adaptation (more aggressive tracking) reduces the average BPP on spatially varying images but increases the complexity of the k update logic and may cause oscillation on noisy images. The current implementation uses the LOCO-I standard adaptation rate, which provides a balanced tradeoff between adaptation speed and stability.

A secondary trade off exists between BRAM line buffer width and maximum supported image width. The current implementation supports up to 4096 pixels per line, consuming a fixed BRAM allocation. Reducing this parameter decreases BRAM usage at the cost of a lower maximum supported resolution.

4.9 Limitations

The current implementation supports 8-bit greyscale input images only. Extension to colour (RGB) images would require either pre-processing the colour channels independently or modifying the MED predictor to exploit inter channel correlation. Additionally, the current design does not implement the exact lossless mode of JPEG-LS, which allows a controlled maximum reconstruction error and can achieve significantly lower BPP values at the cost of not being strictly lossless.

CHAPTER 5

CONCLUSION & FUTURE WORK

5.1 Conclusion

This work presents the design and hardware implementation of lossless image compression algorithm on the Xilinx Zynq 7000 SoC using the PYNQ-Z1 development platform. The proposed architecture employs the Median Edge Detector (MED) for spatial prediction and an adaptive Golomb-Rice entropy coder for efficient binary encoding of prediction residuals. A fully streaming RTL pipeline is realized in Verilog HDL.

The design is packaged as a reusable Vivado IP core and validated on hardware via the PYNQ framework. Compression results on a 12-image greyscale benchmark demonstrate compression ratios from 1.14 to 3.99 (average BPP = 4.74), consistent with the LOCO-I standard. Processing time scales linearly with image size, confirming the $O(N)$ streaming architecture. FPGA resource utilization is low, establishing the viability of the design for deployment on resource-constrained embedded platforms.

In summary, the proposed MEDRICE hardware IP core provides a hardware efficient, low latency lossless image compression solution suitable for integration into real-time embedded imaging pipelines.

5.2 Future Work

Several extensions to the current work are envisioned to enhance its capability and applicability:

Color Image Support: Extend the MED predictor and Rice encoder to process three channel RGB images independently per channel or with inter channel decorrelation to exploit chromatic redundancy.

Near Lossless Mode: Implement the JPEG-LS near-lossless mode, which bounds the maximum reconstruction error to a user-specified value, enabling significantly lower BPP.

Higher Throughput Architecture: Trying multiple pixels per clock architectures to increase throughput beyond 1 pixel per clock, enabling compression of high-resolution video streams in real time.

On-Chip Integration with Image Sensor Interface: Integrate the MEDRICE core directly with a MIPI CSI-2 or parallel CMOS sensor interface IP core to form a complete on chip image acquisition and compression pipeline.

Hardware Decompression Core: Design and implement a corresponding MEDRICE decompressor in RTL to enable full on chip lossless compression and decompression for symmetric communication applications.

Adaptive Context Modeling: Investigate the addition of a context modeling stage based on the LOCO-I context regularization scheme to further reduce the average BPP on challenging high entropy images.

CHAPTER 6

REFERENCES

- [1] **M. J. Weinberger, G. Seroussi, and G. Sapiro**, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Transactions on Image Processing*, vol. 9, no. 8, pp. 1309–1324, Aug. 2000.
- [2] **M. J. Weinberger, G. Seroussi, and G. Sapiro**, "LOCO-I: A low complexity, context-based, lossless image compression algorithm," in *Proc. IEEE Data Compression Conference (DCC)*, Snowbird, UT, USA, 1996, pp. 140–149.
- [3] **ISO/IEC 14495-1:1999**, "Information technology – Lossless and near-lossless compression of continuous-tone still images: Baseline," International Organization for Standardization, Geneva, Switzerland, 1999.
- [4] **R. F. Rice**, "Some practical universal noiseless coding techniques," Jet Propulsion Laboratory, Pasadena, CA, Tech. Rep. JPL-79-22, 1979.
- [5] **S. W. Golomb**, "Run-length encodings," *IEEE Transactions on Information Theory*, vol. 12, no. 3, pp. 399–401, Jul. 1966.
- [6] **M. Abramowitz and C. Morimoto**, "An ASIC implementation of the JPEG-LS lossless image compression standard," in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2001.
- [7] **X. Wu and N. Memon**, "Context-based, adaptive, lossless image coding," *IEEE Transactions on Communications*, vol. 45, no. 4, pp. 437–444, Apr. 1997.
- [8] **A. Kumar and R. Sharma**, "FPGA implementation of LOCO-I lossless image compression on Xilinx Zynq-7000," in *Proc. International Conference on Signal Processing and Communication (ICSC)*, 2019.
- [9] **Xilinx Inc.**, "Zynq-7000 SoC Technical Reference Manual," UG585, v1.12.2, 2018. [Online]. Available: <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>
- [10] **Xilinx Inc.**, "AXI Reference Guide," UG761, v14.3, 2012.
- [11] **Digilent Inc.**, "PYNQ-Z2 Reference Manual," 2019. [Online]. Available: <https://digilent.com/reference/programmable-logic/pynq-z2/reference-manual>

[12] **PYNQ Project**, "PYNQ: Python Productivity for Zynq," 2023. [Online]. Available: <http://www.pynq.io>

[13] **USC-SIPI Image Database**. [Online]. Available: <https://sipi.usc.edu/database/>